HydroSense-Kenya: A Scientific Computing System for Smart Irrigation and Climate-Aware Decision Support

Author: Lead Python Engineer & Senior Scientific Computing Technical Writer
Status: Corporate-Ready Technical Report
Target Audience: Stakeholders, Agronomists, and Engineering Audit Committee

---

1. Executive Summary

HydroSense-Kenya is an integrated scientific computing framework designed to optimize agricultural water use in the Arid and Semi-Arid Lands (ASALs) of Kenya. Operating at the intersection of mathematical modeling, numerical analysis, and decision theory, the system translates physical soil-water interactions into a highly optimized, climate-aware decision-support pipeline.

By transitioning from naive calendar-based scheduling to a vectorized numerical simulation, the system model tracks daily water-mass balances across multiple crop zones. The system architecture leverages: 1. **High-Performance Vectorization & Error Mitigation:** Utilizing NumPy to bypass Python interpreter overhead and manage floating-point arithmetic limitations. 2. **Manual Numerical Solvers:** Custom implementations of root-finding, numerical differentiation, and integration (Simpson's Rule) to calculate exact water allocations and cumulative deficits. 3. **Continuous Dynamics ODE Modeling:** Integrating Runge-Kutta (RK4) and Euler methods to simulate soil moisture depletion curves. 4. **Stochastic Projections:** Deploying Monte Carlo simulations to model precipitation uncertainty over 1,000 distinct climate scenarios. 5. **Rigorous Code Auditing:** Strict adherence to reproducible environments, automated Pytest validation, and responsible AI-assisted programming protocols.

This report details the underlying physical principles, algorithmic justifications, code design, and actionable agricultural insights generated by the system.

---

2. Problem Statement & Scope

Agricultural enterprises in ASAL regions of Kenya face a complex optimization problem. Groundwater and surface water reserves are finite, and the energy required to operate irrigation pumps represents a substantial operational expense. Under-irrigation triggers moisture stress, pushing the soil water volume below the crop's critical wilting point, which leads to permanent stomatal closure, cell

plasmolysis, and severe yield degradation. Over-irrigation saturates the soil past its field capacity, causing rapid gravitational drainage, nutrient leaching, and unnecessary electrical energy consumption.

The central scientific question addressed by HydroSense-Kenya is:

$$\text{Minimize } \sum_{t=1}^{T} I_t \quad \text{subject to} \quad S_t \geq S_{\min} \quad \forall t \in [1, T]$$

Where: * I_t is the irrigation volume applied at day t. * S_t is the volumetric soil moisture percentage. * $S_{\min}$ is the crop-specific minimum allowable moisture threshold (wilting point).

The physical system is governed by the discrete water-mass balance equation:

$$S_{t+1} = S_t + R_t + I_t - ET_t - D_t$$

Where: * R_t is the daily rainfall contribution. * ET_t is the crop evapotranspiration loss, governed empirically by localized temperature (T), wind speed (W), solar radiation ($Solar$), and relative humidity (H):

$$ET = \max(0, 0.12T + 0.35W + 2.4Solar - 0.025H)$$

- D_t is the non-linear drainage representing water lost beyond the soil's field capacity (FC), parameterized by a drainage coefficient (c_d):

$$D_t = c_d \max(0, S_t - FC)$$

---

3. Data Collection & Preprocessing

The system processes three distinct data inputs representing the farm's environmental state and structural parameters: 1. `weather_daily.csv`: Historical records of rainfall, temperature, relative humidity, wind speed, and solar index. 2. `soil_sensor_data.csv`: Daily noon readings of soil moisture, storage tank levels, pump flow rates, and hardware statuses. 3. `crop_zone_parameters.csv`: Agronomic constants for each zone ($S_{\min}$, S_{target}, FC, c_d).

Algorithmic Cleaning Workflow

Raw sensor fields are inherently noisy and prone to hardware failure. The preprocessing pipeline (`src/data_cleaning.py`) processes raw data through the following steps: * **Sensor Fault Masking:** Records displaying a `sensor_status` other than `'OK'` are flagged. Associated physical parameters

are nullified (`np.nan`) to prevent corrupted telemetry from skewing the models. * **Targeted Linear Interpolation:** Missing data is resolved using a grouping-aware linear interpolation method:

$$\hat{y}_t = y_{t_1} + (t - t_1)\frac{y_{t_2} - y_{t_1}}{t_2 - t_1}$$

This is applied strictly within each independent `zone_id` group to prevent cross-contamination of physical properties between different crops. * **Physical Outlier Constraints:** Blatant hardware spikes (e.g., 45.8°C temperatures or 9900 L in a 5000 L tank) are capped using statistical boundaries (95th percentile) or physical volume clamping.

---

4. System Workflow & Code Architecture

HydroSense-Kenya strictly adheres to clean, modular software engineering design patterns, keeping a clear boundary between libraries (`src/`), automated verification scripts (`tests/`), data storage (`data/`), and interactive presentation layers (`notebooks/`).

```
HydroSense-Kenya/
+-- data/raw/ & data/processed/   # Unmodified and cleaned master datasets
+-- notebooks/                    # Levels 1-6 Jupyter Notebooks
+-- src/                          # Modular production-ready code library
|   +-- data_cleaning.py
|   +-- numerical_methods.py      # Custom root, differentiation, integration,
    & linear solvers
|   +-- simulation.py             # ET models, ODE solvers, and Monte Carlo
|   +-- optimization.py           # Secant-based closed-loop irrigation
    scheduling
|   +-- visualization.py
+-- tests/                        # Automated unit tests (Pytest framework)
+-- pytest.ini                    # Standard Python path environment
    configuration
+-- requirements.txt              # Explicit dependency version tracking
```

---

5. Exploratory Data Analysis & Visualizations

To provide actionable intelligence to agronomists, the system generates five core scientific visualizations, transforming raw arrays into interpretable farm conditions:

1. **Climatic Drivers (Rainfall vs. Solar Intensity Overlay):**

- *Interpretation:* Demonstrates a strong inverse relationship. Heavy precipitation events naturally suppress solar intensity due to cloud cover, which in turn suppresses evapotranspiration rates. This validates the inclusion of solar index in the empirical ET model.

2. **Soil Moisture Depletion Curves:**
   - *Interpretation:* Tracks volumetric water content across Zones A, B, and C. Zone C (Maize) approaches its critical wilting point significantly faster than other zones. This is mathematically driven by its higher drainage coefficient ($c_d = 0.22$), indicating a soil profile that retains water poorly and requires highly targeted, frequent irrigation.
3. **Pump Efficiency (Power vs. Flow Scatter Plot):**
   - *Interpretation:* Reveals a linear correlation between power draw and flow rate, but heavily stratified by zone. Zone C requires higher wattage to achieve the same flow rate as Zone A, indicating higher elevation or longer pipe friction losses.
4. **Tank Drawdown & Weather Correlation Heatmap:**
   - *Interpretation:* The drawdown curve shows steady depletion during dry spells. The heatmap confirms a strong negative correlation (-0.68) between humidity and solar index, proving that the ET model must account for these inversely related variables to avoid overestimating water loss on humid days.
5. **Monte Carlo Probability Distribution & Optimization Schedule:**
   - *Interpretation:* The histogram quantifies the risk of water shortage, proving that relying on historical rainfall alone results in an average of 14 stress days. The dual-axis optimization plot visualizes the closed-loop interventions, showing exactly when and how much water the algorithm injects to keep the moisture curve safely above the red wilting-point threshold.

---

6. Methodology & Algorithmic Justification

6.1 Floating-Point Arithmetic & Error Analysis

A critical component of scientific computing is managing the inherent limitations of IEEE 754 floating-point arithmetic. In base-2 binary, fractions like 0.1 and 0.2 have infinite repeating expansions, leading to the well-known Python phenomenon where $0.1 + 0.2 \neq 0.3$.

In a 30-day continuous simulation loop, these microscopic truncation and round-off errors accumulate. If standard equality operators (`==`) are used in root-finding algorithms, the system may enter infinite loops. We mitigate this by strictly utilizing tolerance-based checks (e.g., `np.isclose` with `tol=1e-5`). Furthermore, we modeled the propagation of sensor noise ($\pm 3\%$ Gaussian variance). The simulation proved that unmitigated measurement error propagates directly through the differential equations, causing the system to trigger water pumps

prematurely, wasting energy.

6.2 Vectorization vs. Standard Python Loops

To compute daily evapotranspiration across hundreds of farm zones, standard Python loops introduce severe performance overhead due to the Global Interpreter Lock (GIL).

By utilizing NumPy, math operations are offloaded to pre-compiled C-arrays using SIMD (Single Instruction, Multiple Data) processing. This not only yields speedup factors exceeding **40x to 100x**, but it also standardizes memory allocation to contiguous `float64` blocks, reducing the creation of intermediate Python float objects and slightly mitigating cumulative round-off drift.

6.3 Numerical Differentiation & Integration

To understand the rate of soil-moisture change and the cumulative water deficit, we implemented manual calculus engines from scratch.

Numerical Differentiation: We estimate the daily rate of moisture change ($\frac{dS}{dt}$) using finite differences: * *Forward Difference:* $\frac{S_{t+1}-S_t}{\Delta t}$ (Error: $\mathcal{O}(\Delta t)$) * *Backward Difference:* $\frac{S_t-S_{t-1}}{\Delta t}$ (Error: $\mathcal{O}(\Delta t)$) * *Central Difference:* $\frac{S_{t+1}-S_{t-1}}{2\Delta t}$ (Error: $\mathcal{O}(\Delta t^2)$) Central difference is utilized for interior data points as its quadratic error bound provides a significantly more accurate representation of the true derivative.

Numerical Integration: To calculate the cumulative water deficit over the month ($\int ET(t)dt$), we contrasted two methods: * *Trapezoidal Rule:* Approximates the area under the curve using linear segments. Error is $\mathcal{O}(\Delta t^2)$. * *Simpson's 1/3 Rule:* Approximates the area using parabolic arcs:

$$\int_a^b f(t)dt \approx \frac{\Delta t}{3}\left[f(t_0) + 4f(t_1) + 2f(t_2) + \cdots + f(t_n)\right]$$

Simpson's rule provides a much higher accuracy ($\mathcal{O}(\Delta t^4)$) for smooth, continuous ET curves. Our implementation dynamically handles both even and odd numbers of data points.

6.4 Algorithmic Justification of Root-Finding Solvers

Calculating the exact volume of irrigation water (I) required to reach a target moisture level is complex because the drainage term $D(I)$ is non-linear and non-differentiable at the field capacity boundary. We seek the root of $f(I) = 0$.

- **Bisection Method:** Reliably secure but computationally slow (linear convergence).

- **Newton-Raphson Method:** Achieves quadratic convergence but requires the derivative $f'(I)$. Because the derivative is discontinuous at the field capacity boundary, Newton-Raphson is brittle and can fail from a poor initial guess.
- **Secant Method:** Approximates the derivative using consecutive functional evaluations. It converges at a superlinear rate ($r \approx 1.62$) without requiring analytical derivatives, making it the optimal, robust choice for our piecewise drainage constraint.

6.5 Continuous ODE Modeling: Euler vs. Runge-Kutta (RK4)

Soil moisture depletion is a continuous process. We model it using ordinary differential equations (ODEs): $\frac{dS}{dt} = R(t) + I(t) - ET(t) - D(S)$.

- **Euler Method (First-Order):** Updates linearly. It has a local truncation error of $\mathcal{O}(\Delta t^2)$. It is highly prone to numerical instability, causing simulated moisture to oscillate wildly if Δt is too large.
- **Runge-Kutta Method (RK4, Fourth-Order):** Samples four distinct slope estimates (k_1, k_2, k_3, k_4) across the interval. It has a much smaller local truncation error of $\mathcal{O}(\Delta t^5)$, providing excellent numerical stability for sharp soil-drainage dynamics.

6.6 Stochastic Climate Modeling via Monte Carlo

Deterministic weather data is insufficient for ASAL regions. We run **1,000 Monte Carlo simulations** to generate stochastic rainfall forecasts, perturbing baseline rainfall with Gaussian noise ($\mathcal{N}(0, \sigma^2)$). Routing these through our RK4 solver constructs a probability distribution of crop stress, allowing for probabilistic risk management.

---

7. Responsible AI Validation & Code Audit

HydroSense-Kenya enforces a strict policy regarding AI-assisted programming: **AI is utilized as a syntax assistant, never as an authoritative scientific source.**

All AI-generated outputs were subjected to a rigorous workflow: *Prompt* $\rightarrow$ *Inspect* $\rightarrow$ *Modify* $\rightarrow$ *Validate computationally.* * **Mathematical Correction:** When prompted for an ET formula, AI generated a generic agronomic equation. This was manually overridden and hardcoded to match the specific project requirement ($ET = \max(0, 0.12T + 0.35W + 2.4Solar - 0.025H)$). * **Algorithm Refactoring:** AI-generated RK4 boilerplate code was manually refactored to accept our specific environmental variables (R, I, ET, FC) and route through our custom `soil_water_derivative` function. * **Test Validation:** AI-generated Pytest cases were manually adjusted to include `np.isclose` with a tolerance of `1e-5` to account for the floating-point drift identified in our

error analysis. The final codebase passes 12 automated tests, ensuring absolute scientific integrity.

---

8. Key Findings & Decision Intelligence

We contrasted two distinct irrigation management policies: 1. **Uniform Irrigation Policy:** Applying a fixed 2 mm of water daily. 2. **Predictive Closed-Loop Policy:** Using our Secant root-finding solver to trigger pump operations only when soil moisture approaches the wilting point, irrigating precisely up to the target.

Policy	Total Water Applied	Pump Cycles Triggered	Moisture Stress Days
Uniform	60.0 mm	30	0 days
Closed-Loop	28.4 mm	8	0 days

Agronomic Recommendation: The farm must adopt the **Predictive Closed-Loop Policy**. It achieves **52.6% water savings** and reduces pump activations from 30 to 8. To optimize power usage, pump flow rates should be adjusted to run at low-wattage settings during these 8 targeted windows, avoiding the high-wattage surges associated with emergency dry-soil recovery.

---

9. Strategic Code Snippets

Snippet 1: Vectorized ET Calculation (`src/simulation.py`)

Eliminates loop overhead by processing physical arrays using NumPy vectorized operations, protecting calculations from negative bounds.

```
import numpy as np

def calculate_et_vectorized(T, W, Solar, H):
    """Computes Evapotranspiration using NumPy vectorized operations."""
    et = 0.12 * T + 0.35 * W + 2.4 * Solar - 0.025 * H
    return np.maximum(0, et)
```

Snippet 2: Simpson's 1/3 Rule Integration (`src/numerical_methods.py`)

A manual implementation of numerical integration to calculate cumulative water deficits. It dynamically handles even/odd array lengths to maintain $\mathcal{O}(\Delta t^4)$ accuracy.

```
def simpson_rule(y, dx):
    """Simpson's 1/3 integration rule for cumulative deficit."""
    n = len(y)
```

```
    if n % 2 == 0:
        # Fallback: Simpson for n-1, Trapezoidal for the last segment
        s_integ = (dx / 3) * (y[0] + 4 * np.sum(y[1:-2:2]) + 2 *
            np.sum(y[2:-2:2]) + y[-2])
        trap_integ = 0.5 * dx * (y[-2] + y[-1])
        return s_integ + trap_integ
    return (dx / 3) * (y[0] + 4 * np.sum(y[1:-1:2]) + 2 *
        np.sum(y[2:-2:2]) + y[-1])
```

Snippet 3: Gaussian Elimination with Partial Pivoting (`src/numerical_methods.py`)

Computes water allocation vectors. Partial pivoting protects the system from dividing by zero when dealing with diagonal zeros in the constraint matrix.

```
def gaussian_elimination(A, b):
    """Solves Ax = b using Gaussian elimination with partial pivoting."""
    n = len(b)
    M, v = A.copy().astype(float), b.copy().astype(float)

    for i in range(n):
        max_row = np.argmax(abs(M[i:n, i])) + i
        if M[max_row, i] == 0: raise ValueError("Matrix is singular.")

        M[[i, max_row]], v[[i, max_row]] = M[[max_row, i]], v[[max_row, i]]

        for j in range(i + 1, n):
            factor = M[j, i] / M[i, i]
            M[j, i:] -= factor * M[i, i:]
            v[j] -= factor * v[i]

    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (v[i] - np.dot(M[i, i+1:], x[i+1:])) / M[i, i]
    return x
```

Snippet 4: Runge-Kutta 4 (RK4) Step Solver (`src/simulation.py`)

Tracks continuous soil-water changes by sampling four intermediate slopes across a single step interval (Δt), ensuring numerical stability.

```
def rk4_step(S_t, dt, R, I, ET, field_capacity, drainage_coeff):
    """Performs a single high-stability 4th-order Runge-Kutta step."""
    k1 = soil_water_derivative(S_t, R, I, ET, field_capacity, drainage_coeff)
    k2 = soil_water_derivative(S_t + 0.5*dt*k1, R, I, ET, field_capacity,
        drainage_coeff)
    k3 = soil_water_derivative(S_t + 0.5*dt*k2, R, I, ET, field_capacity,
        drainage_coeff)
```

```
    k4 = soil_water_derivative(S_t + dt*k3, R, I, ET, field_capacity,
        drainage_coeff)

    return S_t + (dt / 6.0) * (k1 + 2*k2 + 2*k3 + k4)
```

Snippet 5: Secant-Root-Finding Optimizer (`src/optimization.py`)

Uses our custom Secant solver to calculate the exact irrigation volume (I) needed to return soil moisture back to target levels.

```
def optimize_irrigation_schedule(initial_moisture, rainfall, et, min_moisture,
    target_moisture, field_capacity, drainage_coeff):
    """Calculates exact water needs using a Secant-based closed-loop
        controller."""
    # ... initialization ...
    for t in range(n_days):
        S_pred = rk4_step(moisture_record[t], dt, rainfall[t], 0,
            et[t], field_capacity, drainage_coeff)

        if S_pred < min_moisture:
            def moisture_deficit_function(I):
                S_next = rk4_step(moisture_record[t], dt,
                    rainfall[t], I, et[t], field_capacity,
                    drainage_coeff)
                return S_next - target_moisture

            opt_I, _ = secant(moisture_deficit_function, 0.0,
                target_moisture - S_pred)
            irrigation_record[t] = max(0, opt_I)
        else:
            irrigation_record[t] = 0.0

        moisture_record[t+1] = rk4_step(moisture_record[t], dt,
            rainfall[t], irrigation_record[t], et[t], field_capacity,
            drainage_coeff)

    return moisture_record, irrigation_record
```